Framework::
→ Set of standards and rules for developing automation code

Different types of frameworks::
① Linear scripting framework
② Modular testing framework
③ Data-driven framework
④ Keyword-driven framework

python frameworks::
① pytest
② unittest
③ Robot framework

Advantages::
① Reusability
② Scalable
③ Logs / Defects
④ Easy to setup / cleanup
⑤ Easy to integrate with version control tools [bit bucket, CI-CD
⑥ Easy maintain

pytest::
→ It is the one of python framework to create and execute testcases
→ we need to install pytest and import it in our test script

```
import pytest
```

Syntax::

```
pip install pytest
```

→ Test script name should start with test underscore (test_)
→ All testcases implemented as python functions / classes
To implement testcase in .py file::
def test_sample():
→ pytest testcase doesn't require any regular arguments
→ To execute pytest testcase → pytest script.py
→ what are the test functions that starts with test_ will be executed automatically whenever we call script with pytest
→ Normal functions will be ignored and only the functions created as pytest testcases will be executed

→ In a single test script (test_testcase.py) we can define 'n' no. of testcases as a functions.
→ To run particular testcase from a pytest testscript
↳ pytest test.py :: test_2
↳ To run multiple → pytest test.py :: test_2, test_3

→ pytest support different features
① Defining the testcases as pytest function as classes
② To execute all the test suit - individual testcases
③ we can change the order of test execution
④ we can run multiple test cases in parallel
⑤ pytest supports fistures, markers and hwoks
⑥ using pytest we can generate test reports in different formats Ex:: html, xml
⑦ we can run multiple test scripts from a single folder at a time

Syntax ::
pytest -s -v test_sample2.py:: test_sample1
To run multiple testcases

fixture ::
fixtures are special functions in pytest which are used for setup and cleanup activities
→ we need to decorate a normal function with a special decorator called @pytest.fixture to make a nrml function as a pytest fixture

Ex:: import pytest
#setup & cleanup code

@pytest.fixture
def fixture1():
    print("sample fixture")

def test_sample1(fixture1):
    print("In sample1 TC")

→ we use "yield" keyword in the fixtures to seperate setup and cleanup code.

→ order of testcase execution::

① setup code
② Actual testcase
③ cleanup code

Ex::

```
import pytest
@pytest.fixture
def fixture1():
    print("setup code")   #setup code
    yield
    print("cleanup code")  #cleanup code
def test_sample1(fixture1):
    print("In sample1 TC")
```

Ex::

```
list1=[]
#TC1:: push some data to list1 and verify len is even or odd
#TC2:: push some nums to list1 and find largest number from list1
```

Setup
  Add some values to TC

TC
  verify len is even or odd

cleanup
  Remove all elements from list1

Ex::

```
import pytest
list1=[]
@python.fixture
def fixture1():
    #setup
    print("Adding elements to list1")
    list1.extend([1,2,3,4,5,6])
    print(list1)
    yield
    print("Removing all elements from list1")
    list1.clear()
    print(list1)
def test_verify_len(fixture1):
    print("verify len of list")
```

```
if len(list1)%2==0:
    print ("even")
else:
    print ("odd")

def test_max_num(fixture1):
    print("verifying max num")
    print (max (list1))
```

→ "function" - fixture will be called for every testcase (function)

→ "class" - fixture will be called once per testclass (shared by all methods)

→ "module" - fixture will be called once per module (shared by all tests in the file)

→ "package" - fixture will be called once per package

→ "session" - fixture will be called once per test session (shared across all tests)

→ "Default scope of fixture is "function."

Ex:: import pytest

```
@pytest.fixture(scope="session")
def fixture1():
    print("setup code")   # setup code
    yield
    print("cleanup code") #cleanup code

def test_sample1(fixture1):
    print("In sample1 TC1")

def test_sample2(fixture1):
    print("In sample1 TC2")
```

(scope="function", autouse=True)
↓
No need to add fixture1 in test_sample1 & 2

Autouse = True :: fixture will be called automatically for all the testcases

→ no need to pass fixture as argument

Test reports ::

we can generate html, xml or json test reports after the testcase execution

→ pip install pytest-html [To Generate html reports]

To generate test reports

→ pytest -s -v test_sample6.py --html=sample_report.html

Assert::
→ It is used to compare actual result with expected result
→ If it not matches it will throw "Assertion Error"

Ex::
```
import pytest
cluster=1
def test_01():
    assert cluster>2
        #assert 10+10==30,"Invalid output"
        #assert False
```

Ex::
```
def test_02():
    assert 10+0==30,"Invalid output"

def test_03():
    if cluster>2:
        assert False
```

Markers::
→ Markers are used to control the flow of execution of pytest testcases
Features::
① we can run only specific testcases using markers
② we can skip some of the testcases
③ we can change the order of execution
④ we can mark tests as regression, sanity, smoke etc...
⑤ we can test the testcases with multiple inputs [unit testing]
⑥ we can define our custom markers
⑦ skip based on specific condition

→ we can mark the testcases by using inbuilt decorator
@pytest.mark

Ex:: To change the order of pytest execution
→ pip install pytest-order
```
import pytest
@pytest.mark.order(3)
def test_01():
    Print("Test1")
```

```
@pytest.mark.order(1)
def test_02():
    print("Test2")

@pytest.mark.order(2)
def test_03():
    print("Test3")
```

Skip :: To skip testcases
Skipif :: To skip testcases based on specific condition

```
import pytest
import sys
@pytest.mark.skip(reason="BUG 001, In open State")
def test_01():
    print("Test1")

@pytest.mark.skipif(sys.platform=="win32", reason=
                    "unsupported platform")

def test_02():
    print("Test2")

def test_03():
    print("Test3")
```

pytest.ini :: used to create custom markers

```
[pytest]
markers =
    regression :: To run regression testcases
    sanity     :: To run sanity testcases
    smoke      :: To run smoke testcases
```

⇒ to mark the testcases with custom markers

```
import pytest

@pytest.mark.regression
def test_01():
    print("Test1")

@pytest.mark.sanity
def test_02():
    print("Test2")
```

```
@pytest.mark.smoke
def test_03():
    print("Test3")

# To run custom markers
pytest -s -v script.py -m markername
```

---

parametrize :: To test our testcases with multiple inputs

Ex::

```
import pytest
@pytest.mark.parametrize("input,expected",[
  (1,2),
  (2,4),
  (3,10),
])
def test_double(input, expected):
    assert input * 2 == expected
```

---

conftest.py ::

→ It is one of the configuration file in pytest
→ we will define all the fixtures inside conftest.py file
→ this file will be automatically executed whenever we call pytest testcase - script
→ we can define multiple ~~testcases~~ fixtures within the conftest file and we need to save this file in same folder where we create the testcases

requirement.txt ::

→ used to install the test dependencies
→ pip

Ex::

```
paramiko == "3.5.1"
pytest == "8.0.3"
numpy >= "10.0"
```

```
pip install -r requirement.txt
```

→ To run multiple tests in parallel

```
pytest -xdist
```

conftest.py → filename ← conftest example

```
import pytest
@pytest.fixture
def setup():
    return "Test setup Done"
```

fix10.py → filename

```
import pytest
@pytest.mark.order(2)
def test_01(setup):
    print("test1", setup)

@pytest.mark.order(3)
def test_02(setup):
    print("test2", setup)

@pytest.mark.order(1)
def test_03(setup):
    print("test3", setup)
```

---

conftest.py ::

→ It is one of the configuration file where we defined shared fixtures

→ It is automatically discovered by pytest so we don't need to import it

→ we can define multiple fixtures in this file and reuse them across the testcases

→ It should be placed in the same folder, where testcases are present